

Runtime Storage Organization: Stack

Lecture 19

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value  $v$ , and partitions  $a[m..n]$  so that
        $a[m..p-1]$  are less than  $v$ ,  $a[p] = v$ , and  $a[p+1..n]$  are
       equal to or greater than  $v$ . Returns  $p$ . */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

top

main

bottom

integer a[11] (global)

Frame for main

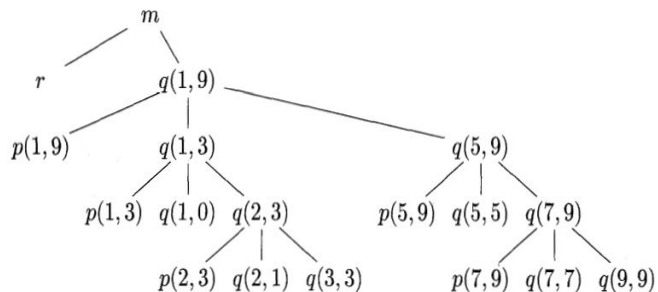
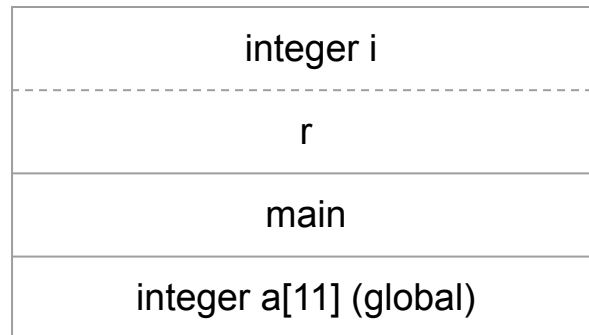
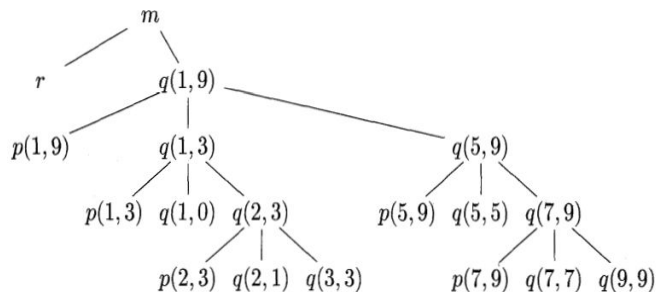


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value  $v$ , and partitions  $a[m..n]$  so that
        $a[m..p-1]$  are less than  $v$ ,  $a[p] = v$ , and  $a[p+1..n]$  are
       equal to or greater than  $v$ . Returns  $p$ . */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



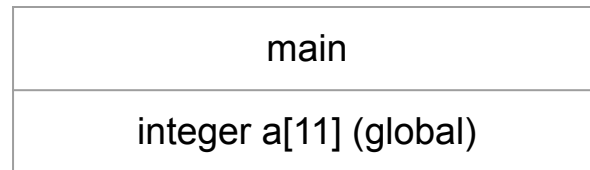
r is activated

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value  $v$ , and partitions  $a[m..n]$  so that
        $a[m..p-1]$  are less than  $v$ ,  $a[p] = v$ , and  $a[p+1..n]$  are
       equal to or greater than  $v$ . Returns  $p$ . */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



return from r so r has been popped

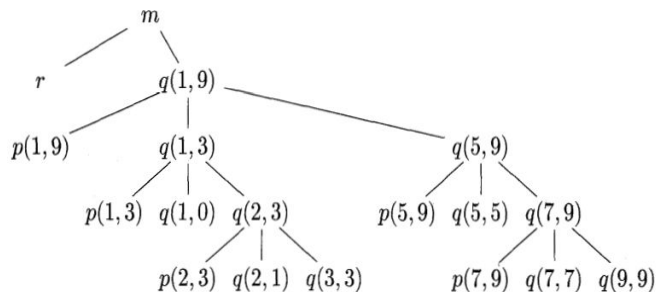


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

integer i
q(1,9)
integer m,n
main
integer a[11] (global)

call quicksort(1,9) so q(1,9) pushed

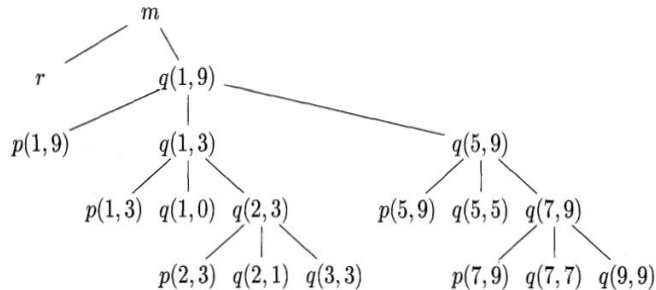


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

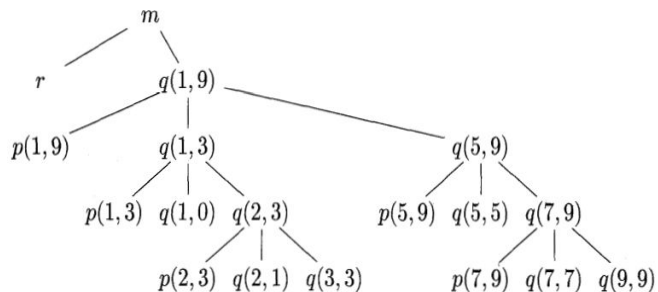


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

p(1,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call partition(1,9) so p(1,9) pushed

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

integer i
q(1,9)
integer m,n
main
integer a[11] (global)

return from p so p(1,9) has been popped

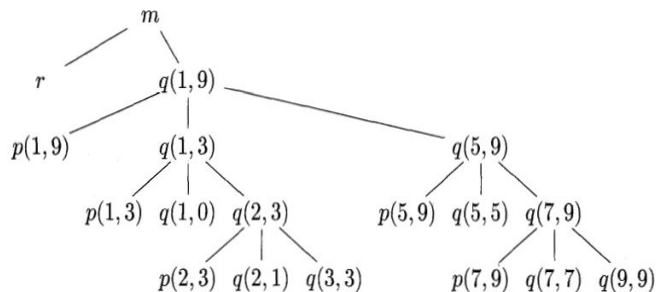
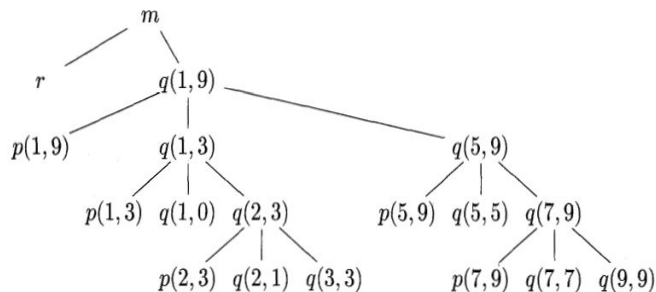


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

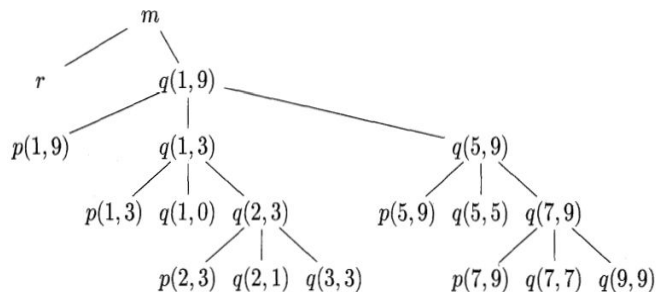
Call quicksort(1,3) so q(1,3) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



p(1,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

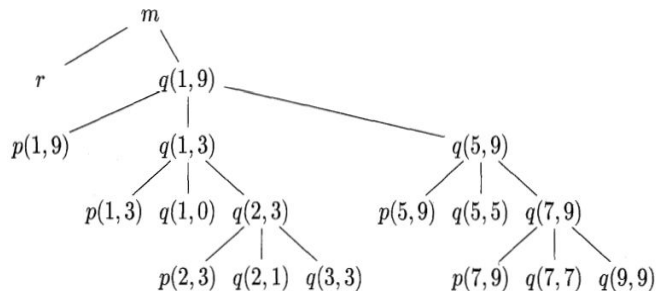
inside q(1,3), call partition(1,3) so p(1,3)
pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

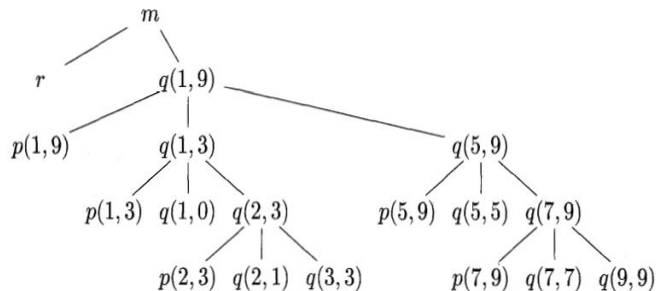
return from p so p(1,3) has been popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(1,0)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

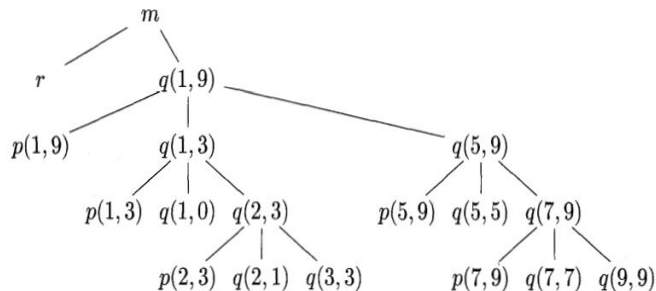
Call quicksort(1,0) so q(1,0) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

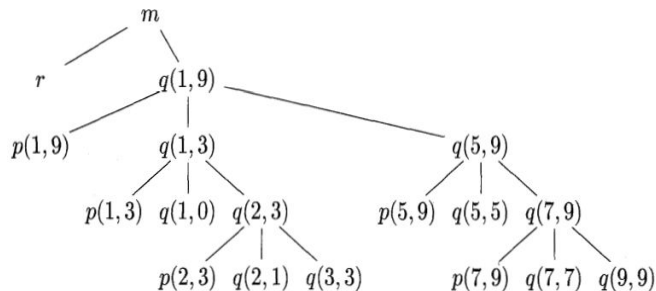
return from q(1,0) so q(1,0) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(2,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call quicksort(2,3) so q(2,3) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

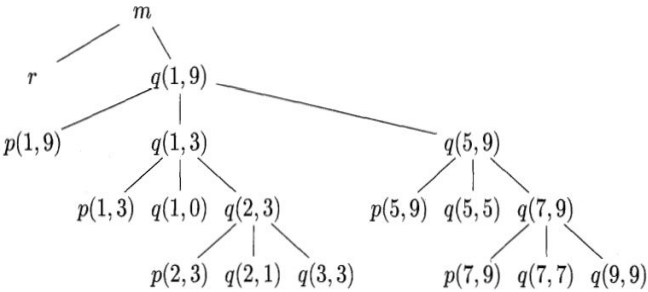


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

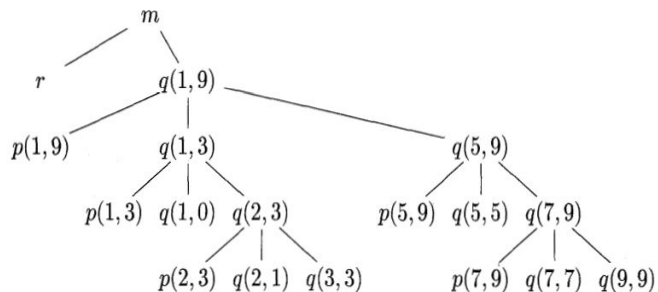
p(2,3)
integer m,n
integer i
q(2,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call partition (2,3) so p(2,3) pushed

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

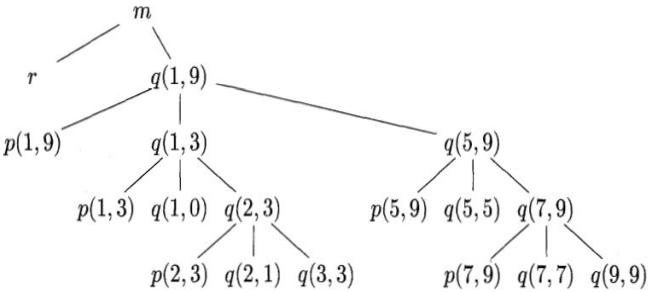


integer i
q(2,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

return p(2,3) so p(2,3) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

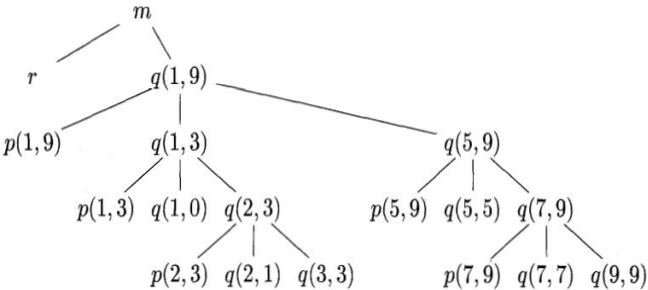


integer i
q(2,1)
integer m,n
integer i
q(2,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

call quicksort(2,1) so q(2,1) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

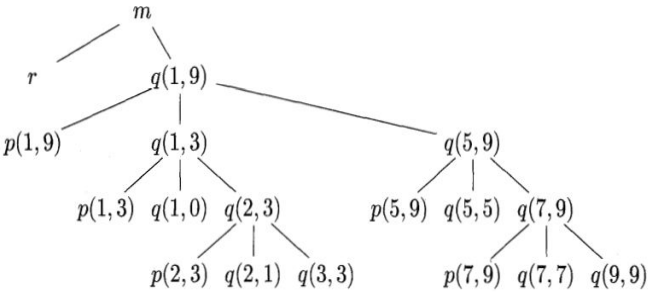


integer i
q(2,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Return q(2,1) so q(2,1) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

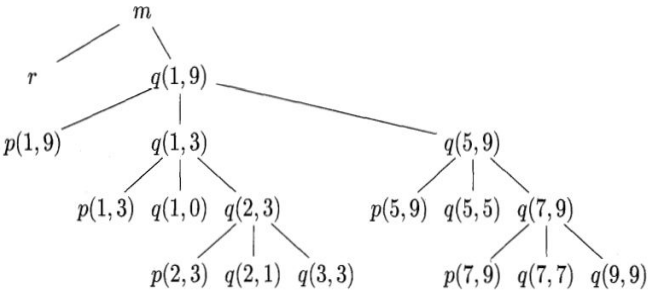


integer i
q(3,3)
integer m,n
integer i
q(2,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

call quicksort(3,3) so q(3,3) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```



integer i
q(2,3)
integer m,n
integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

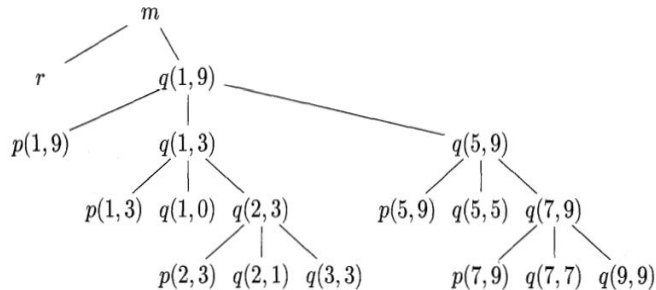
Return q(3,3) so q(3,3) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(1,3)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

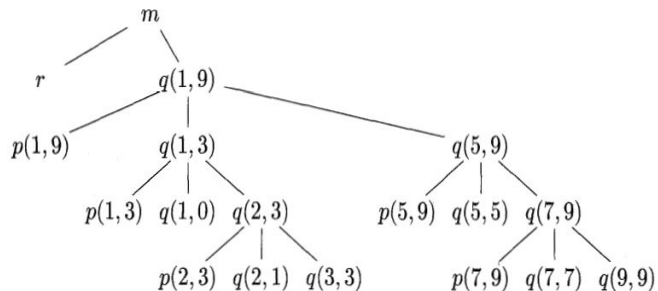
Return q(2,3) so q(2,3) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(1,9)
integer m,n
main
integer a[11] (global)

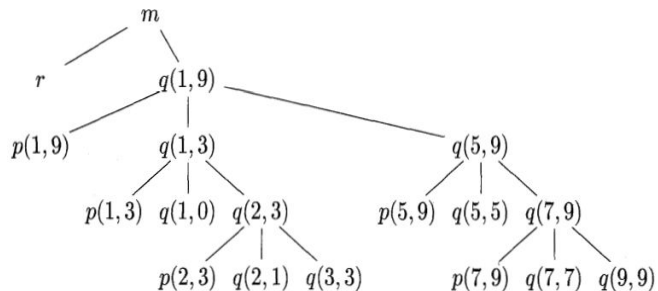
Return q(1,3) so q(1,3) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

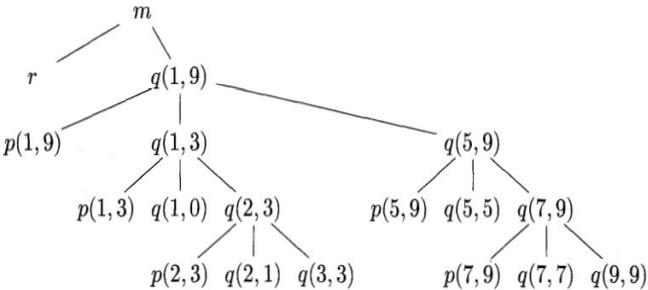


integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call quicksort(5,9) so q(5,9) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```



p(5,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call partition(5,9) so p(5,9) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Return partition(5,9) so p(5,9) popped

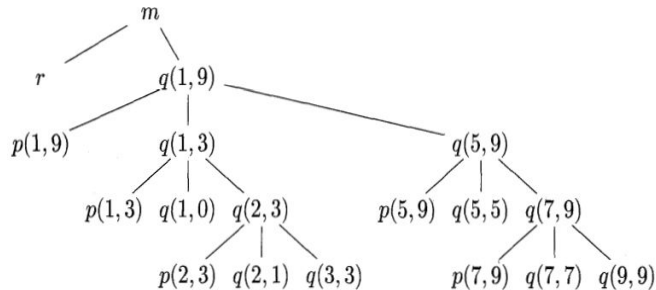
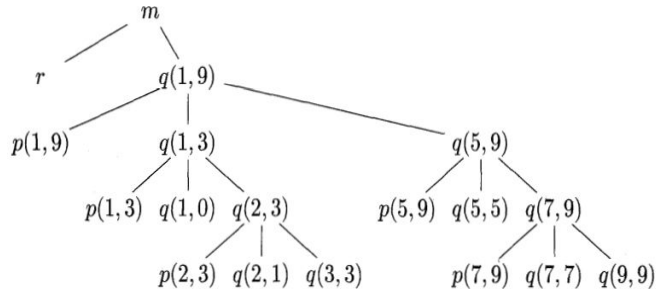


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(5,5)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

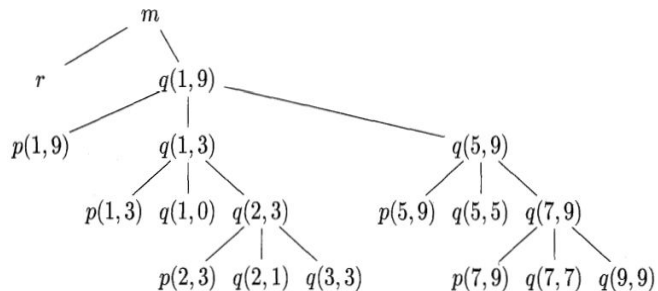
Call quicksort(5,5) so q(5,5) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

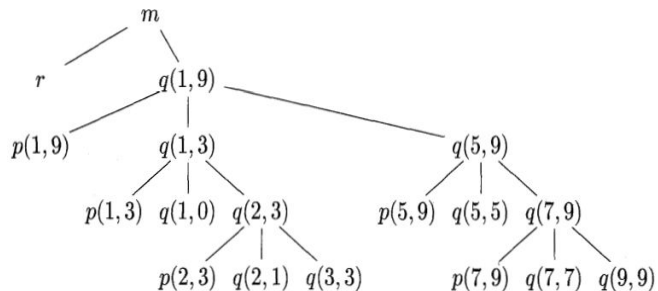
Return quicksort(5,5) so q(5,5) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```



integer i
q(7,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call quicksort(7,9) so q(7,9) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

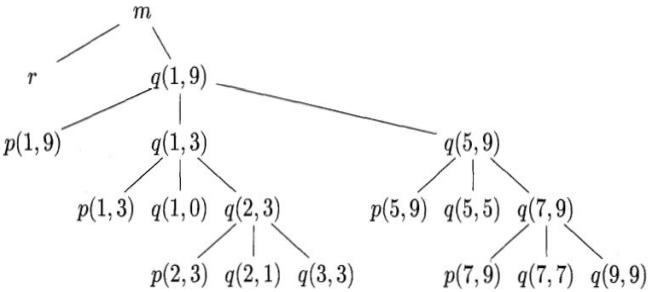
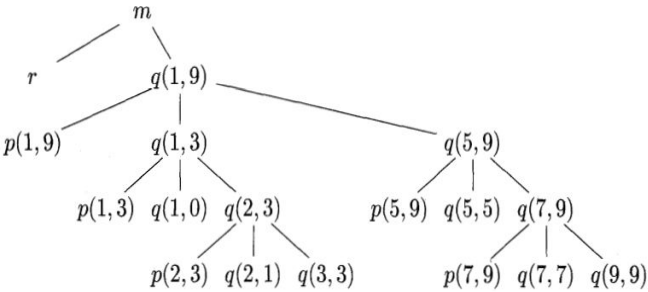


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

p(7,9)
integer m,n
integer i
q(7,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call partition(7,9) so p(7,9) pushed

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

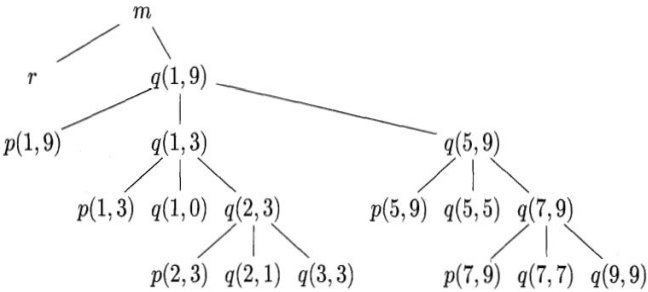


integer i
q(7,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Return partition(7,9) so p(7,9) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

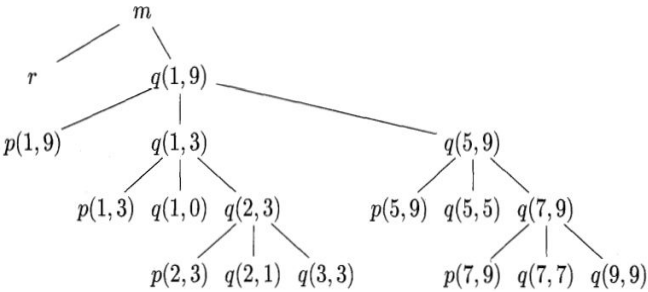


integer i
q(7,7)
integer m,n
integer i
q(7,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call quicksort(7,7) so q(7,7) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

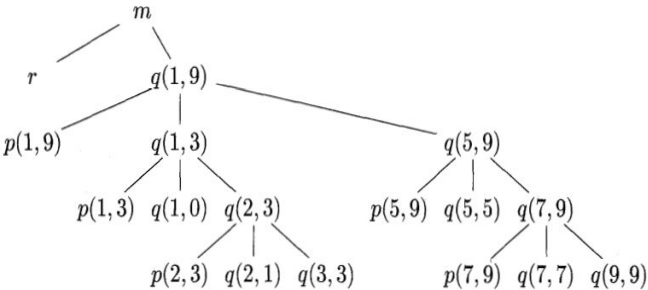


integer i
q(7,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Return quicksort(7,7) so q(7,7) popped

Figure 7.4: Activation tree representing calls during an execution of quicksort

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

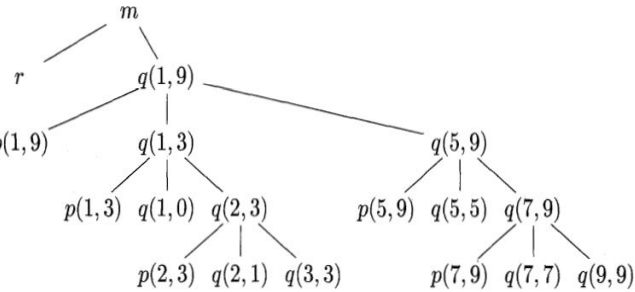


integer i
q(9,9)
integer m,n
integer i
q(7,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Call quicksort(9,9) so q(9,9) pushed

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```
int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```



integer i
q(7,9)
integer m,n
integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Return quicksort(9,9) so q(9,9) popped

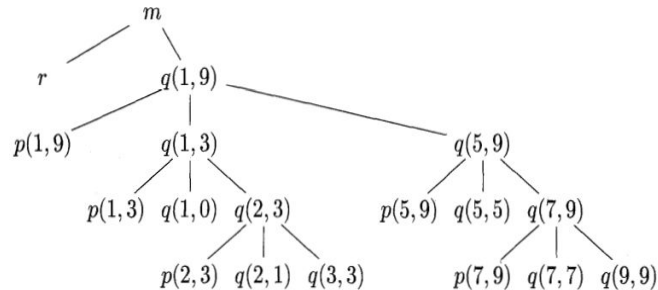
Figure 7.4: Activation tree representing calls during an execution of quicksort

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

integer i
q(5,9)
integer m,n
integer i
q(1,9)
integer m,n
main
integer a[11] (global)



Return quicksort(7,9) so q(7,9) popped

Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

integer i
q(1,9)
integer m,n
main
integer a[11] (global)

Return quicksort(5,9) so q(5,9) popped

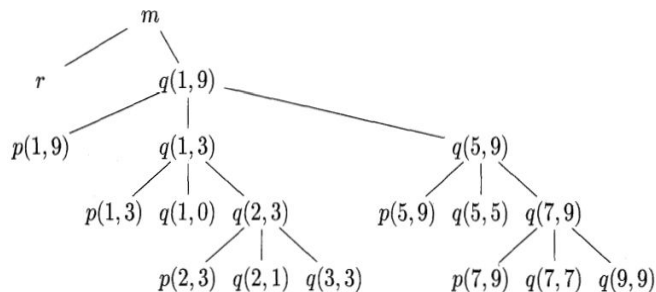


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value  $v$ , and partitions  $a[m..n]$  so that
        $a[m..p-1]$  are less than  $v$ ,  $a[p] = v$ , and  $a[p+1..n]$  are
       equal to or greater than  $v$ . Returns  $p$ . */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

main
integer a[11] (global)

Return quicksort(1,9) so q(1,9) popped

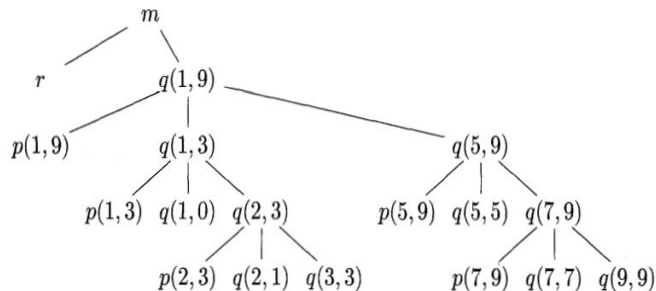


Figure 7.4: Activation tree representing calls during an execution of *quicksort*

```

int a[11];
void readArray() { /* Reads 9 integers into a[1],...,a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value  $v$ , and partitions  $a[m..n]$  so that
        $a[m..p-1]$  are less than  $v$ ,  $a[p] = v$ , and  $a[p+1..n]$  are
       equal to or greater than  $v$ . Returns  $p$ . */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}

```

integer a[11] (global)

return from main

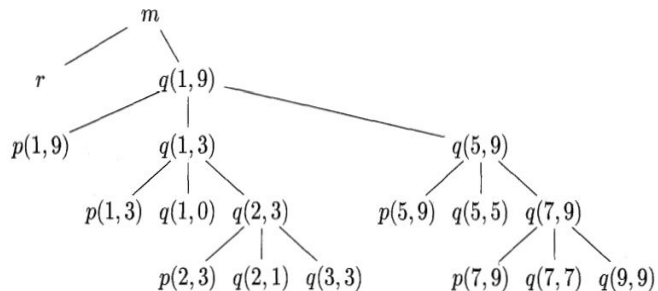


Figure 7.4: Activation tree representing calls during an execution of *quicksort*